We mentioned Richard Bellman in chapter 1, but here we will discuss the Bellman equation, which underpins much of reinforcement learning. The Bellman equation shows up everywhere in the reinforcement learning literature, but if all you want to do is write Python, you can do that without understanding the Bellman equation. This section is optional; it's for those interested in a bit more mathematical background.

As you'll recall, the Q function tells us the value of a state-action pair, and value is defined as the expected sum of time-discounted rewards. In the Gridworld game, for example, $Q_\pi(s,a)$ tells us the average rewards we will get if we take action a in state s and follow policy π from then forward. The optimal Q function is denoted Q^* and is the Q function that is perfectly accurate. When we first start playing a game with a randomly initialized Q function, it is going to give us very inaccurate Q value predictions, but the goal is to iteratively update the Q function until it gets close to the optimal Q^*.

The Bellman equation tells us how to update the Q function when rewards are observed,

$$Q_\pi(s_t,a_t) \leftarrow r_t + \gamma \times V_\pi(s_{t+1}),$$

where $V_\pi(s_{t+1}) = max[Q_\pi(s_{t+1},a)]$

So the Q value of the current state, $Q_\pi(s_t,a)$, should be updated to be the observed reward r_t plus the value of the next state $V_\pi(s_{t+1})$ multiplied by the discount factor γ (the left-facing arrow in the equation means "assign the value on the right side to the variable on the left side"). The value of the next state is simply whatever the highest Q value is for the next state (since we get a different Q value for each possible action).

If we use neural networks to approximate the Q function, we try to minimize the error between the predicted $Q_\pi(s_t,a_t)$ on the left side of the Bellman equation and the quantity on the right side by updating the neural network's parameters.